ProjectSelectionModal UI設計書

メタデータ

- **コンポーネントID**: ProjectSelectionModal
- **コンポーネント名**: プロジェクト選択モーダル
- **ファイルパス**: `frontend/src/components/ExperimentWorker/ProjectSelectionModal.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

1. 概要

1.1 コンポーネントの目的

実験作業者が作業対象のプロジェクトを選択するためのモーダルダイアログです。選択可能な状態のプロジェクト一覧をテーブル形式で表示し、検索・フィルタリング機能を提供します。

1.2 主要機能

- プロジェクト一覧のテーブル表示
- ラジオボタンによる単一選択
- あいまい検索機能（プロジェクト名、プロジェクトコード）
- プロジェクト名フィルター
- 選択解除機能
- ページネーション

1.3 使用場所

- ExperimentRecordTab（実験記録タブ）

- ProgressCheckTab（進捗確認タブ）
- InstructionConfirmationTab（指示確認タブ）

---

2. コンポーネント構成

2.1 画面イメージ



2.2 レイアウト構造

```
┌─Modal──────────────────────────────────────────────────┐
│ Header: [FolderOutlined] プロジェクト選択                  │
├────────────────────────────────────────────────────────┤
│ Body:                                                  │
│ ┌──────────────────────────────────────────────────┐   │
│ │ フィルタリングエリア                                  │   │
│ │ ┌──────────┬──────────┐                          │   │
│ │ │[検索BOX] │[プロジェクト名]│                        │   │
│ │ │  400px   │    200px    │                        │   │
│ │ └──────────┴──────────┘                          │   │
├─┴──────────────────────────────────────────────────┤   │
│ │ Table (高さ: 350px、スクロール可能)                  │   │
│ │ ┌──────┬──────────┬────────┬────────┐            │   │
│ │ │[○]   │プロジェクト名│プロジェクトCD│状態│            │   │
│ │ │      │ 250px    │ 150px  │ 120px │            │   │
│ │ ├──────┼──────────┼────────┼────────┤            │   │
│ │ │[○]   │Project A │A-001   │実行中  │            │   │
│ │ │[●]   │Project B │B-002   │実行中  │(選択中)     │   │
│ │ │[○]   │Project C │C-003   │一時停止│            │   │
│ │ └──────┴──────────┴────────┴────────┘            │   │
│ │ Pagination: [1] [2] [3] ... 合計X件              │   │
│ └──────────────────────────────────────────────────┘   │
├────────────────────────────────────────────────────────┤
│ Footer:                                                │
│ [選択解除] ──────────────── [キャンセル] [決定]          │
│  (danger)                          (default) (primary) │
└────────────────────────────────────────────────────────┘

モーダル幅: 900px
テーブル高さ: 350px（スクロール可能）
```

2.3 使用コンポーネント

Ant Design コンポーネント

- `Modal` - モーダルダイアログ
- `Table` - プロジェクト一覧テーブル
- `Input` - あいまい検索入力
- `Select` - プロジェクト名フィルター
- `Button` - 選択解除、キャンセル、決定ボタン
- `Space` - 要素間隔制御
- `Row`, `Col` - フィルターエリアのレイアウト
- `Typography.Text` - テキスト表示
- `theme` - デザイントークン取得
- `Spin` - ローディングスピナー（空データ時のローディング表示）

アイコン

- `FolderOutlined` - プロジェクトアイコン
- `CheckOutlined` - 決定アイコン
- `CloseOutlined` - 選択解除アイコン
- `SearchOutlined` - 検索アイコン

カスタムコンポーネント

- なし（Ant Designコンポーネントのみ使用）

---

3. Props

```
interface ProjectSelectionModalProps {
  open: boolean;                                // モーダルの開閉状態
```

```
  onClose: () => void;                          // 閉じるイベントハンドラ
  selectedProject: string;                      // 現在選択中のプロジェクトID
  onProjectChange: (value: string | null) => void;  // プロジェクト変更イベント
  projects: WorkerProject[];                    // プロジェクト一覧（API取得済みデータ）
  loading: boolean;                             // データ読み込み中フラグ
}
```

---

4. State（内部状態）

```
// 一時選択状態（決定ボタン押下時に確定）
const [tempProject, setTempProject] = useState<string>(selectedProject);

// フィルター状態
const [searchText, setSearchText] = useState<string>('');
const [selectedProjectName, setSelectedProjectName] = useState<string | undefined>(undefined);
```

4.1 状態のリセットタイミング

モーダルが開かれた時（`open === true`）に以下をリセット：

- `tempProject` ← `selectedProject`（現在の選択状態を復元）
- `searchText` ← `''`（検索テキストをクリア）
- `selectedProjectName` ← `undefined`（プロジェクト名フィルターをクリア）

---

5. テーブルデータ

5.1 データ型定義

```
interface ProjectTableData {
  key: string;                  // 行キー（projectId）
  projectId: string;            // プロジェクトID
  projectName: string;          // プロジェクト名
  projectCd?: string;           // プロジェクトコード
```

```
  projectStatus: number;         // プロジェクトステータス（数値：コードマスター参照）
  isCurrentlySelected: boolean;  // 現在選択中フラグ
}
```

5.2 データソース

- **ソース**: Props `projects`（`WorkerProject[]`）。APIから取得済みのデータを親コンポーネントが渡す
- **フィルター条件**: API側で選択可能なプロジェクトのみ返却されるため、コンポーネント側でのフィルタリングは不要
- **ローディング状態**: Props `loading` が `true` の間はテーブルにローディングスピナーを表示し、フィルター入力も無効化される

5.3 テーブルカラム定義

カラム	dataIndex	幅	配置	説明
プロジェクト名	projectName	250px	left	FolderOutlinedアイコン付き、太字表示
プロジェクトCD	projectCd	150px	left	プロジェクトコード表示
状態	projectStatus	120px	center	コードマスター（`project_status`）から取得したラベルを表示

---

6. フィルタリング機能

6.1 あいまい検索

対象フィールド:

- プロジェクト名（`projectName`）
- プロジェクトコード（`projectCd`）

検索ロジック:

```
const matchesSearch = !searchText ||
  item.projectName.toLowerCase().includes(searchText.toLowerCase()) ||
  (item.projectCd && item.projectCd.toLowerCase().includes(searchText.toLowerCase()));
```

6.2 プロジェクト名フィルター

- **種類**: 完全一致フィルター
- **選択肢**: テーブルデータから重複を除いた一意のプロジェクト名リスト
- **ソート**: アルファベット順

```
const projectNameOptions = useMemo(() => {
  const uniqueNames = Array.from(new Set(allTableData.map(item => item.projectName)));
  return uniqueNames.sort().map(name => ({ label: name, value: name }));
}, [allTableData]);
```

6.3 フィルタリング適用

すべてのフィルター条件はAND条件で適用されます。

```
const filteredData = useMemo(() => {
  return allTableData.filter(item => {
    return matchesSearch && matchesProjectName;
  });
}, [allTableData, searchText, selectedProjectName]);
```

---

7. イベント

7.1 イベント一覧

イベント名	トリガー	処理内容
handleOk	決定ボタンクリック	tempProjectを親に通知してモーダルを閉じる

handleCancel	キャンセルボタンクリック	tempProjectをリセットしてモーダルを閉じる
handleClearSelection	選択解除ボタンクリック	tempProjectを空文字に設定
rowSelection.onChange	テーブル行選択	tempProjectを更新
setSearchText	検索入力変更	検索テキストを更新（即座にフィルタリング）
setSelectedProjectName	プロジェクト名選択	プロジェクト名フィルターを更新

7.2 イベント詳細

handleOk（決定）

```
const handleOk = () => {
  onProjectChange(tempProject || null);  // 空文字の場合はnullを渡す
  onClose();
};
```

handleCancel（キャンセル）

```
const handleCancel = () => {
  setTempProject(selectedProject);  // 元の選択状態に戻す
  onClose();
};
```

handleClearSelection（選択解除）

```
const handleClearSelection = () => {
  setTempProject('');  // 選択を解除（空文字）
};
```

行選択

```
const rowSelection = {
  type: 'radio' as const,  // ラジオボタン（単一選択）
  selectedRowKeys: tempProject ? [tempProject] : [],
  onChange: (selectedRowKeys: React.Key[]) => {
    setTempProject(selectedRowKeys[0] as string);
  },
};
```

---

8. スタイリング

8.1 モーダルスタイル

```
<Modal
  open={open}
  onCancel={handleCancel}
  width={900}  // モーダル幅
  title={...}
  footer={...}
>
```

8.2 ヘッダースタイル

```
<Space>
  <FolderOutlined style={{ color: token.colorPrimary }} />
  <Text strong>{t('experimentWork.projectSelection.title')}</Text>
</Space>
```

8.3 フッタースタイル

```
<Space style={{ width: '100%', justifyContent: 'space-between' }}>
  {/* 左側: 選択解除ボタン */}
  <Button danger icon={<CloseOutlined />} disabled={!tempProject}>
    選択解除
```

```
  </Button>

  {/* 右側: キャンセル・決定ボタン */}
  <Space>
    <Button>キャンセル</Button>
    <Button type="primary" icon={<CheckOutlined />}>決定</Button>
  </Space>
</Space>
```

8.4 テーブルスタイル

```
<Table
  columns={columns}
  dataSource={filteredData}
  rowSelection={rowSelection}
  pagination={{
    pageSize: 10,
    showSizeChanger: true,
    showTotal: (total) => `合計: ${total} 件`,
    pageSizeOptions: ['10', '20', '50', '100'],
    showQuickJumper: true,
  }}
  scroll={{ y: 350 }}  // 縦スクロール
  size="middle"
  locale={{
    emptyText: loading ? <Spin /> : 'プロジェクトがありません'
  }}
/>
```

---

9. レスポンシブ設計

9.1 フィルターエリアのレスポンシブ

```
<Row gutter={16}>
  <Col flex="400px">  {/* 検索ボックス */}
    <Input ... />
  </Col>
  <Col flex="200px">  {/* プロジェクト名フィルター */}
```

```
    <Select ... />
  </Col>
</Row>
```

推奨:

- デスクトップ: 横一列配置
- タブレット: 検索ボックスを全幅、フィルターを2列
- モバイル: すべて縦1列

現状:

- flex指定による固定幅レイアウト
- レスポンシブブレークポイントは未設定

---

10. ページネーション

10.1 設定

```
pagination={{
  pageSize: 10,                    // 1ページあたり10件
  showSizeChanger: true,           // ページサイズ変更可能
  showTotal: (total) => `合計: ${total} 件`,
  pageSizeOptions: ['10', '20', '50', '100'],
  showQuickJumper: true,           // ページ番号入力可能
}}
```

10.2 推奨ページサイズ

データ量	推奨ページサイズ
~20件	10
~100件	20
~500件	50
500件~	100

11. 多言語化

11.1 使用する翻訳キー

```
// タイトル・ボタン
t('experimentWork.projectSelection.title')
t('experimentWork.projectSelection.clearSelection')
t('experimentWork.projectSelection.cancel')
t('experimentWork.projectSelection.confirm')
```

11.2 ハードコードされている文字列（翻訳キー未適用）

以下の文字列は現状ハードコードされています（将来的に翻訳キー化推奨）：

```
// 検索・フィルタープレースホルダー
placeholder="プロジェクト名またはコードで検索"
placeholder="プロジェクト名でフィルタ"

// ページネーション
showTotal: (total) => `合計: ${total} 件`

// 空データ時のメッセージ
emptyText: 'プロジェクトがありません'

// テーブルカラムヘッダー
'プロジェクト名', 'プロジェクトCD', '状態'
```

11.3 ステータス表示

ステータスのラベルは翻訳キーではなく、コードマスター（`useCodeMaster('project_status')`）から取得します。

```
const { options: projectStatusOptions } = useCodeMaster('project_status');
// projectStatus（数値）→ label（文字列）に変換して表示
```

```
const label = projectStatusOptions.find(opt => opt.value === projectStatus)?.label ??
String(projectStatus);
```

---

12. アクセシビリティ

12.1 キーボード操作

- Tabキーでフォーカス移動
- Enterキーで決定ボタン実行
- Escapeキーでキャンセル
- テーブル行はクリックまたはラジオボタンで選択

12.2 スクリーンリーダー対応

- Modal titleでダイアログの目的を明示
- Table columnsでカラム名を明示
- ボタンのアイコンとテキストを併用

---

13. パフォーマンス考慮事項

13.1 useMemoの使用

以下の計算をメモ化してパフォーマンスを最適化：

```
// テーブルデータ生成
const allTableData = useMemo(() => {...}, [projects, selectedProject]);

// プロジェクト名選択肢
const projectNameOptions = useMemo(() => {...}, [allTableData]);

// フィルタリング適用
const filteredData = useMemo(() => {...}, [allTableData, searchText, selectedProjectName]);
```

13.2 最適化のポイント

1. **データはAPI取得済み**: 親コンポーネントがAPIからデータを取得して渡すため、コンポーネント内でのデータ取得は不要
2. **メモ化**: 計算コストの高い処理（フィルタリング、選択肢生成）をuseMemoで最適化
3. **仮想スクロール**: Table内部でAnt Designが提供（scroll設定）
4. **ページネーション**: 大量データを分割表示

---

14. 備考

14.1 特記事項

1. **一時選択状態**
   - テーブルで選択した内容は `tempProject` に保存
   - 決定ボタン押下時のみ親コンポーネントに通知
   - キャンセル時は元の選択状態に戻る
2. **選択可能なプロジェクト**
   - API側でフィルタリング済みのデータが渡されるため、コンポーネント側での判定は不要
   - 表示されるプロジェクトはすべて選択可能
3. **選択解除機能**
   - プロジェクトの選択を解除できる
   - ボタンは選択中の場合のみ有効（ `disabled={!tempProject}` ）
   - 決定時に `null` が親に渡される
4. **フィルターのクリア**
   - モーダルを開くたびにフィルターがリセットされる
   - ユーザーが前回のフィルター状態に惑わされない

14.2 将来の拡張

1. ステータスフィルターの追加
2. 作成日・更新日でのソート
3. お気に入り機能
4. 最近選択したプロジェクトの履歴表示
5. プロジェクトのプレビュー表示
6. 複数選択モード
7. フィルター状態の永続化
8. レスポンシブ対応の強化（ブレークポイント設定）

14.3 関連ドキュメント

- ExperimentRecordTab UI設計書
- ProgressCheckTab UI設計書
- InstructionConfirmationTab UI設計書

---

15. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-19	初版作成	Cline
1.1.0	2026-03-11	ソースコードとの差分修正（Props追加、テーブルカラム変更、所有者フィルター削除、データソースをAPI取得に変更、コードマスター方式のステータス表示に変更）	Cline